NANYANG TECHNOLOGICAL UNIVERSITY

School of Electrical and Electronic Engineering

Final Year Project Interim Report

Modular vs. End-to-End Vision-Language-Action Models for Embodied Robotic Manipulation

Project code	A1005-261
Student	Tay Juen Jie
Matriculation no.	U2322880A
Supervisor	Assoc Prof Cheah Chien Chern
Laboratory	Robotics I (S2-B6a-01)
Date	10 November 2026

1. Introduction

1.1 Background

A person should be able to tell a robot arm what to do in plain speech. Two families of systems now try to do this. The modular family puts a large language model (LLM) in front of pretrained perception and scripted motion primitives [2]. The end-to-end family trains one vision-language-action (VLA) model that maps camera images, robot state and an instruction straight to motion [6].

Each family reports results on its own robot, tasks and data, so a reader cannot tell how much of a difference comes from the architecture. Shan and Cheah [1] built a modular LLM system on a UR5e for power cable manipulation. They argue that the modular design gives flexibility beyond a single VLA model. No VLA model was run as a baseline in that work, so the claim is still untested. This project tests it on real hardware.

1.2 Aim

The aim is to compare a modular LLM pipeline (Architecture A) with an end-to-end VLA model (Architecture B) on the same UR5e cell. The user speaks a command, for example *"place that orange into that red container"*, and the arm carries it out. The deliverable is the comparison. It reports where each architecture succeeds, how each fails, and what each costs in data, compute, money and engineering effort.

1.3 Objectives

Each objective has a condition that can be checked. Table 1 lists the objectives, their conditions and their status on 29 October 2026.

ID	Objective	Done when	Status
T1	Calibrated cell	Camera to base transform fitted, then checked by a reach test, with the error in mm	Done 21 Oct 2026. Reach error 2 to 7 mm after correction (Section 6.4)
T2	Architecture A on Tier 0	At least 70% success over 20 trials, spoken input	First full pick and place on 21 Oct 2026. 20-trial run planned for December 2026
T3	Demonstration dataset	At least 50 real episodes per task that load in LeRobot	Recorder and exporter written. Recording starts after the exams
T4	Architecture B on Tier 0	At least 50% success over 20 trials, behind the safety envelope	Robot side written. Training starts in January 2027
T5	Evaluation harness	Frozen task suite, per-trial log, imports neither architecture	Done 11 Aug 2026
T6	Comparison result	Both architectures run through the harness on every attempted tier	Planned for February 2027

Table 1. Technical objectives and their status.

1.4 Scope

Every result comes from the real UR5e. Simulation was used only as scaffolding before the semester, and no simulated result is reported. Mobile manipulation, multiple arms, learned grasp planning, training a VLA from scratch and training a speech model are out of scope. Speech recognition is an off-the-shelf model used as it is.

1.5 Purpose of this report

This report records the work of Semester 1 and the plan for Semester 2. Architecture B is reported as a plan, because its demonstrations are recorded after the exams. Section 3 sets out how the comparison is

kept fair. Section 4 explains Architecture A, and Section 5 explains Architecture B. Section 6 reports the work so far, and Section 7 gives the plan.

2. Related Work

Table 2 places this project among the works it builds on. Code as Policies [2] marks where the project is today, since a language model composes pretrained perception and scripted motion with nothing trained. AgenticLab [4] is the closest work to the completed project. It runs a vision-language agent on a UR5e in a closed loop of grounding, planning, execution, verification and replanning. It also compares its agent with a π0.5 VLA fine-tuned on 30 to 40 demonstrations, and the VLA often grasped the wrong object. That comparison is one experiment in a benchmark of agents. This project makes it the main study, with a frozen protocol, tiers for ambiguity and for mid-task change, a stage for every failure, and the cost of each side.

Two works frame the ambiguity experiment. KnowNo [3] showed that an LLM planner can tell when it is unsure and ask a person for help. This project does not claim the idea of asking. It measures the cost of not asking, because Architecture B has no output that can form a question. BayesianVLA [5] gives published evidence for what B will do instead. Its authors found that VLA models often learn a shortcut from the image alone and ignore the instruction, and that this fails when one scene fits several tasks, as in Tier 2.

Work	What it does	Limitation or role for this project
Shan and Cheah [1]	GPT-4o agent with a bank of five skills on a UR5e, applied to power cable manipulation	Claims modular flexibility beyond a VLA, but runs no VLA baseline. Closed-vocabulary detector trained on 30 images
Code as Policies [2]	An LLM writes programs that call perception and control functions	No training needed. The current stage of Architecture A
KnowNo [3]	Conformal prediction tells an LLM planner when to ask for help	Credits prior work on asking. This project measures the cost of not asking
AgenticLab [4]	Closed-loop vision-language agent on a UR5e. Also beats a fine-tuned π0.5 VLA on two tasks	Closest work to the completed project. Its VLA test is a side experiment, while here the comparison is the main study
BayesianVLA [5]	Shows VLAs can ignore the instruction and act on the image alone	Backs the prediction for B in the ambiguous tier
SmolVLA [6]	Compact VLA of about 450 million parameters that outputs chunks of 50 actions	The model fine-tuned for Architecture B

Table 2. Related works and how this project uses them.

3. Method: A Fair Comparison

3.1 System overview

Figure 1 shows both architectures on one cell. A shared speech front end, faster-whisper large-v3 behind push-to-talk, turns the spoken command into one transcript. Both architectures receive that same transcript. Architecture A is an LLM agent, Claude Sonnet 5.5, that loops through planning, perceiving and acting with one tool per turn. Its perception chain is OWLv2 [7] for open-vocabulary detection, SAM [8] for an object mask, and depth from the fixed camera for an object position in the robot base frame. Architecture B is SmolVLA [6]. An encoder path reads two camera views, the robot state and the transcript, and a flow-matching path turns noise into a chunk of 50 small changes of the tool pose. Both send every motion through the same safety envelope and the same controller. A snowflake marks a pretrained part used as it is. A flame marks a part trained on our demonstrations, and only Architecture B has one.

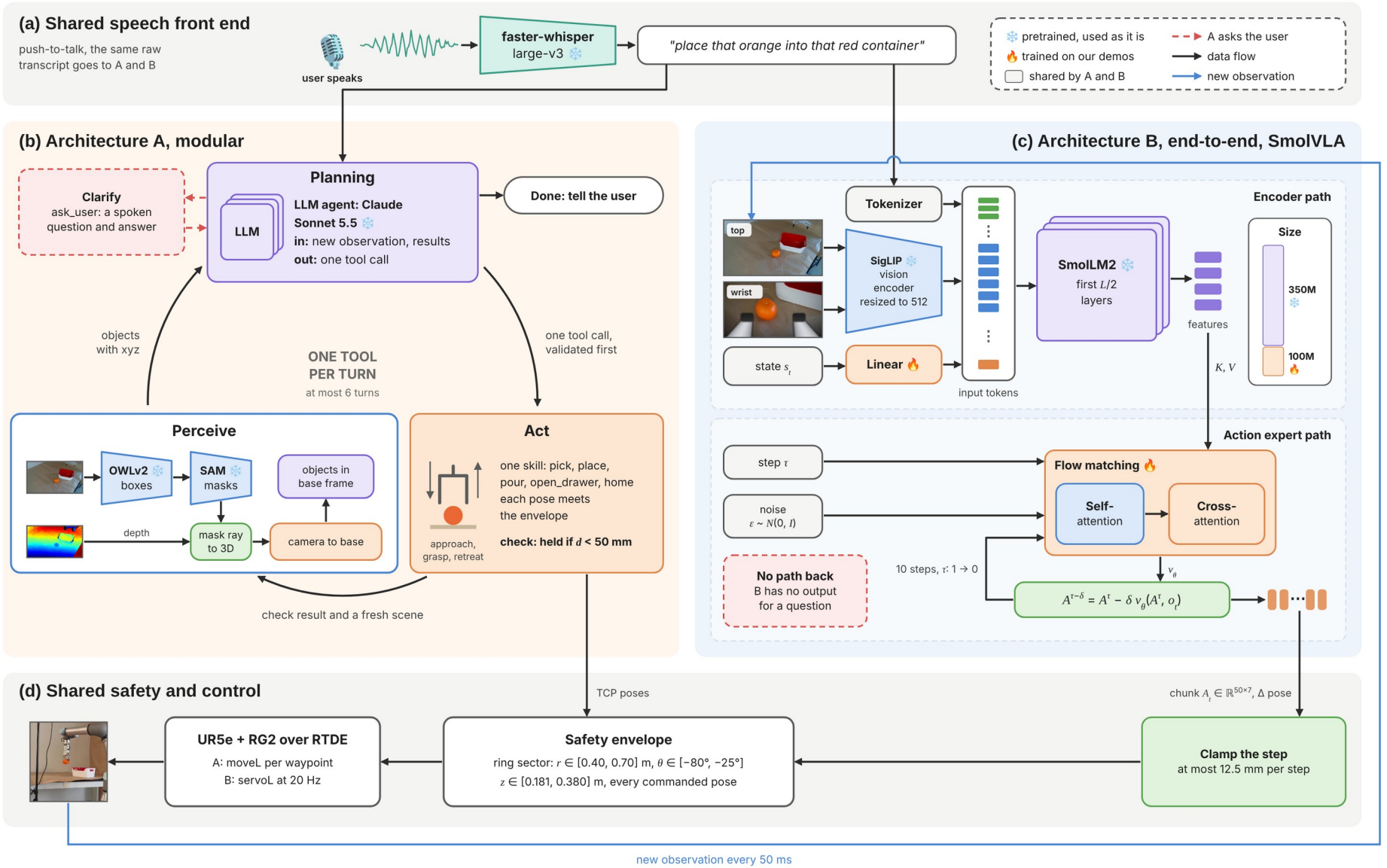


Figure 1. Both architectures on the same cell. (a) One speech front end gives both the same transcript. (b) Architecture A loops through planning, perceiving and acting, one tool per turn. (c) Architecture B runs an encoder path and a flow-matching path to output an action chunk. (d) Both pass through the same envelope and controller.

3.2 Fairness in comparison

The result is only worth something if both architectures get the same problem. Five things are held the same, and each is enforced in code (Table 3).

Control	What is the same	How it is enforced
Same cell	One UR5e, one OnRobot RG2 gripper, one fixed RGB-D camera, one table	Every tunable value lives in one configuration file
Same shared stages	One speech front end, one controller, one safety envelope	Code is shared only if both architectures use it at run time. A test checks the imports
Same action space	A change of tool pose in the base frame plus a gripper target, seven numbers	B's step limit is derived from A's speed ceiling, and a test checks that they agree
Same frozen tests	Tiers, spoken commands, 20 layouts on five fixed crosses and success rules	The suite file is frozen. Any change gets a dated entry in its changelog and is declared in the report
Blind harness	Trials run through one call, run(instruction) to outcome	The harness imports neither architecture, and a test enforces it

Table 3. The five controls that keep the comparison fair.

3.3 Known differences

Two differences cannot be removed. Table 4 states them and how each is handled, so that a reader can weigh them.

Difference	Why it exists	How I handle it
Extra wrist camera	Reference SmolVLA setups use an overhead view plus a wrist view. A uses depth from the fixed camera, which B cannot read	Each side gets the sensors its design needs. Nothing depends on wrist depth, and the report states the bias
Prior knowledge vs demos	A carries knowledge in its pretrained LLM, its detector and its hand-written skills. B learns from about 50 demonstrations per task	Both costs are logged and shown side by side: A's hours, lines of code and API cost, and B's episodes, lab hours and GPU hours

Table 4. Known differences between the architectures.

3.4 Task suite and predictions

The task suite has five tiers (Table 5). Each architecture runs 20 trials per tier. Objects are placed on five fixed crosses on the table, the same five for every tier, and each object keeps one fixed orientation. A layout is one way of placing the objects on the crosses. Tier 0 uses all 20 ways to place two objects on five crosses. Tiers 1 and 2 use 20 of the 60 ways to place three objects, chosen so each object stands on each cross four times. Both architectures get the same 20 layouts in the same order. B's demonstrations and A's tuning stay off the crosses, so the scored positions are never seen before scoring. The spoken commands are scripted, and half of them are phrasings never used in training or in prompts. Each trial logs the transcript, so a speech error is never blamed on A or B.

Tier	Task	Example command	Prediction
0	Static pick and place	"Place that orange into that red container"	Both succeed. Checks the wiring only
1	Pouring	"Pour the orange juice into the glass"	B may win. The tilt is hard to script
2	Ambiguous command	"Pour me a drink" with two drinks on the table	A should win. It can ask, and B falls back on its training data
3	Dynamic drawer	"Put the snack in the drawer", and a person closes the drawer mid-task	B may win. It reacts at 20 Hz without a plan
4	Generalisation	Objects and phrasings never seen before	A should win. Its detector is open-vocabulary

Table 5. Task tiers and the prediction for each.

In short, A should win on ambiguity and on traceability. Every failure of A is logged against a stage, such as speech, grounding, detection, depth, transform, planning or execution. A failure of B has no stage to point to. B may win on motion that is hard to script, such as a pour or a drawer that closes again.

4. Architecture A: Modular LLM Agent

4.1 From a pipeline to a loop

On 7 October 2026 I decided to change how Architecture A works. In the D1 design, A was a fixed pipeline with a planner at the end. It perceived the table once, and the LLM then planned every step before any step ran. A review of that design against the task suite found three problems.

1. A clarification answer could not be acted on. After the user says "orange juice", the pipeline cannot look again for that object, because its word list was fixed at the start. This breaks Tier 2.
2. The planner could chain several skills in one reply. It saw the result of a pick only after the place had already run.
3. Nothing checked a step after it ran. A failed grasp went unnoticed, and Tier 3 needs exactly that check.

The reason for the change is the fairness of the result. A comparison is only as strong as its weaker side. Recent modular systems act one step at a time and verify each step [4]. If A stayed a dated pipeline, B could win against a weak baseline, and an examiner could dismiss the result for that reason. I made the change before any trial was recorded, so no result was lost.

4.2 Perception, planning and skills

The current A is a loop of plan, perceive and act, with one skill per turn (Figure 2). A run starts with grounding. The LLM turns the transcript into short noun phrases such as "orange" and "red tray", because the detector takes phrases and not sentences.

Perception turns the camera frames into object positions in the robot base frame. OWLv2 [7] finds a box for each phrase, and SAM [8] cuts a mask of the object inside the box. A ray through the mask centre, scaled by the depth of the object's own pixels, gives a point in the camera frame. The camera to base transform (Section 6.4) maps that point into the base frame. An object outside the safety envelope is reported as out of reach.

The planner is Claude Sonnet 5.5. Each turn it sees the transcript, the detected objects and every earlier result, and it returns one tool call. The call is checked against its schema, the envelope and the known objects before it runs, and a refused call goes back to the planner with the reason. The planner can also ask the user a spoken question. A run ends when the planner says it is done, or after six turns.

Each skill is a scripted motion. A pick moves the tool centre point (TCP) through three poses: an approach pose 12 cm above the pick position, a grasp pose at the object and a retreat pose above it. A place uses the same three poses over the place position. After each pick or place the table is perceived again, and the pick check reports the object held if it lies within 50 mm of the TCP. A failed step does not end the run, and the planner may retry. Every stage writes a trace record with pass or fail, duration and data, so each failure has a stage.

This version ran on the arm on 21 October 2026. It fixes the second and third problems in Section 4.1. The first is only partly fixed, because the noun phrases are still chosen once at the start (Section 7.1).

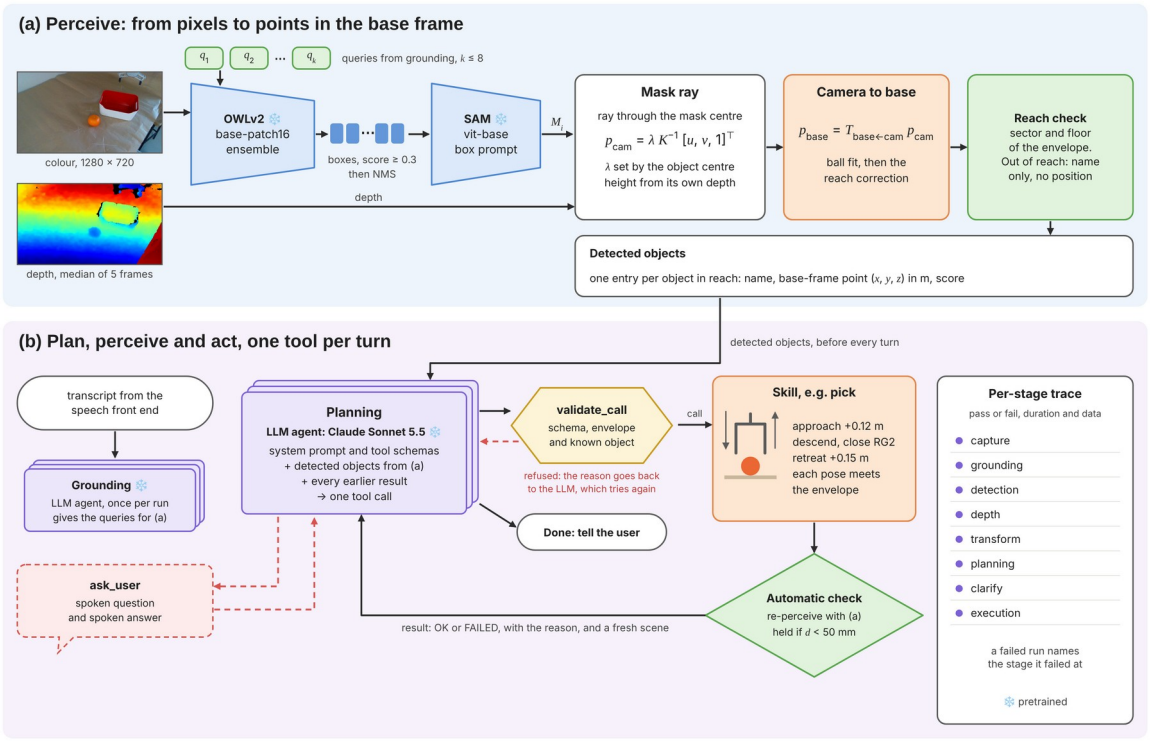


Figure 2. Architecture A, the version that ran on 21 October 2026. (a) Perception turns pixels into points in the robot base frame. (b) The planner calls one tool per turn, and each motion is checked.

5. Architecture B: Fine-tuned SmolVLA

5.1 Design

Architecture B is SmolVLA [6], fine-tuned on demonstrations recorded on this cell (Figure 3). An encoder path reads two camera views, the robot state and the transcript. SmolVLA keeps its vision-language model frozen by default and trains the action expert and the state projection. The action expert uses flow matching. Starting from noise, ten Euler steps follow a learned velocity field to a chunk of 50 actions. Each action is a change of tool pose in the base frame plus a gripper target, the shared action space of Table 3.

Inference runs on LeRobot's own policy server and robot client, so the arm keeps moving while the model computes the next chunk [9]. B has no path back to the user, because it has no output that can form a question.

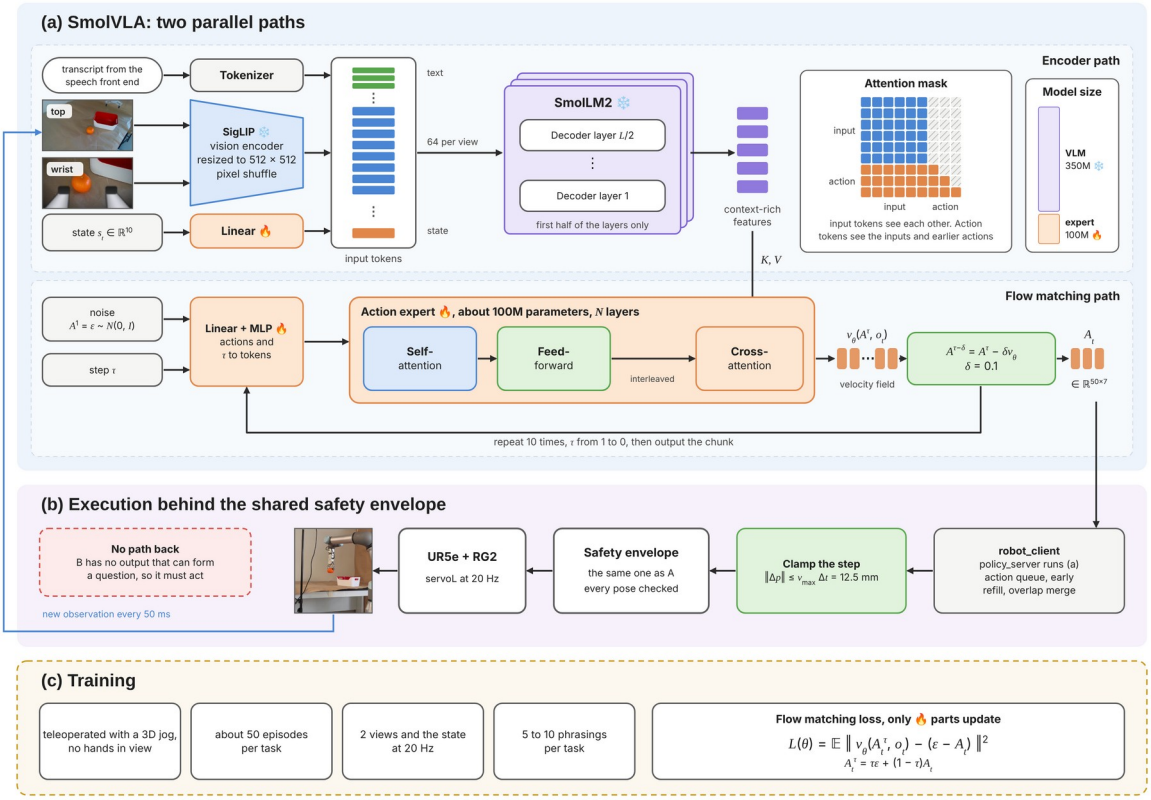


Figure 3. Architecture B. (a) The encoder path turns two views, the state and the instruction into features. The flow-matching path predicts a velocity field, and ten Euler steps turn noise into a chunk of 50 actions. (b) Each action is clamped and checked before the arm moves. (c) Training starts in Semester 2, and only the parts marked with a flame are trained.

5.2 Training plan

Training starts after the Semester 1 exams, and the plan follows a fixed order.

- Record teleoperated demonstrations on the UR5e with a 3D jog device, so that no hands appear in the camera views
- Record about 50 episodes per task
- Log two camera views and the robot state at 20 Hz
- Use 5 to 10 phrasings per task, so that held-out phrasing can be tested
- Start with Tier 0 in a small workspace
- Fine-tune SmolVLA on the recorded episodes
- Widen the workspace once Tier 0 works

5.3 Safety on the arm

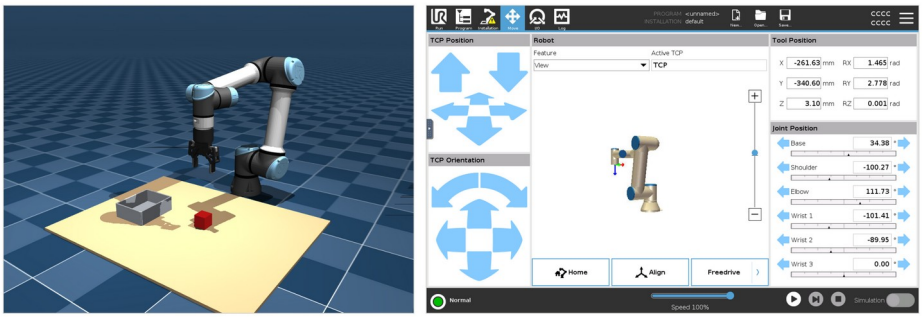
Safety comes before the first run on the arm. Every predicted action is clamped to the shared speed ceiling and checked against the same envelope as A. A model under an unfamiliar view can output a chunk that moves the arm hard, and the envelope stops it.

6. Work Done So Far

6.1 Before the semester

I started on this project on 28 June 2026, six weeks before the official start. That time went to the parts that need no lab access. I also completed the **UR e-Series Core Track** from Universal Robots Academy, which covers the setup, programming and safety settings of e-Series arms such as the UR5e.

Two simulators served as scaffolding (Figure 4). URSim, the UR controller simulator, stood in for the arm while I wrote the control stack. This covered the RTDE and URScript interfaces, a custom RTDE recipe for joint logging, a dashboard client and the controller class. MuJoCo stood in for the cell while I wrote the data path: the demonstration recorder, the episode store, the converter to the LeRobot dataset format and the hand-eye module. OWL-ViT base gave correct boxes at a confidence near 0.008 with the wrong colour labels, so I chose OWLv2 instead. I also found that the recorder counted simulation steps instead of the clock and recorded at about 4 Hz. A wall-clock gate fixed it at 20 Hz. Both simulators were retired once the real cell was in reach.



(a) MuJoCo **(b)** URSim

Figure 4. Simulation used before the semester. (a) MuJoCo with the UR5e and Robotiq 2F-85 models. (b) URSim running the PolyScope interface.

6.2 Software in Weeks 1 to 5

From 10 August to 13 September I split the code into Architecture A, Architecture B and a shared layer, with a test that guards the boundary. I wrote Architecture A in full, the speech front end, and the robot side of Architecture B on LeRobot's policy server and robot client. I froze the evaluation harness on 11 August, before either pipeline was tuned. The unit test suite now has 331 passing tests, and none of them needs the arm.

6.3 Grasp failures and diagnosis

The first picks on the real arm, on 1 and 13 October, all missed (Figure 5). Three observations narrowed the search, and they pointed to two causes.

01 **The gripper closed behind the object.** The fingers came down past the far side of the orange.

02 **The misses were all similar.** A random error would scatter. A common offset points to a systematic error.

03 **The logs showed correct detections.** OWLv2 found the orange in the right place in the image, so the fault came after the detector.

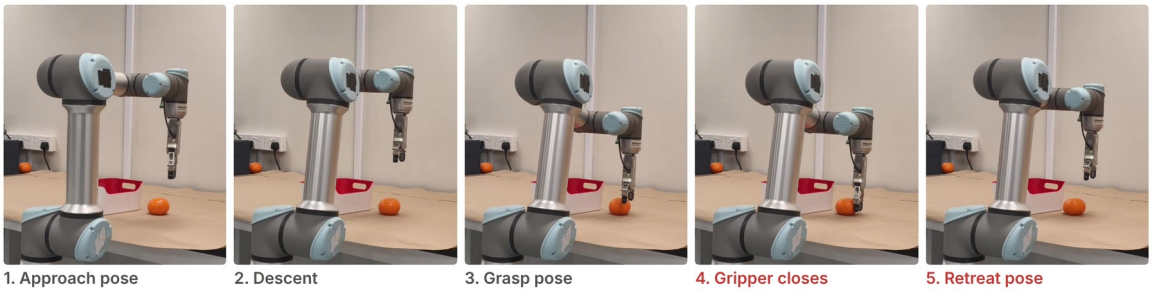


Figure 5. A failed pick on the real arm before the fixes, from phone video. The fingers close behind the orange.

The first cause was a wrong depth point. The depth of the object was the average of the depth values inside the detection box. A round object does not fill its box, so the corners hold table pixels. With the

camera looking down at an angle, those pixels lie farther away than the object. The average moved the pick position along the camera ray to behind the orange (Figure 6). The fix uses SAM [8] to cut a mask of the object inside the box, and the pick position now comes from the mask, so it lies on the object.

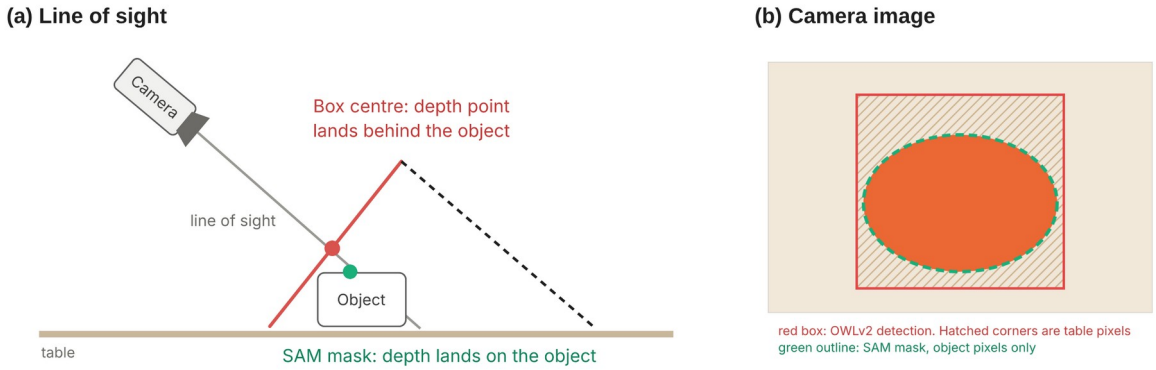


Figure 6. Why the box depth put the target behind the object. (a) Along the camera line of sight, with the face of the detection box the camera sees in red. (b) The camera image: the detection box holds table pixels that the SAM mask leaves out.

The second cause was a bias in the camera to base transform. Section 6.4 explains how the transform is fitted. A shift that is the same at every fitting pose does not raise the fit residual, so the residual cannot reveal it. The fix is a reach test with keyboard jog, run after every calibration fit. The arm hovers 2 cm above a thin pen cross on the table, and I jog it with the keyboard until the TCP is over the cross. The TCP position is then read over RTDE, and its offset from the commanded position is the reach error.

6.4 Calibration without markers

The cell uses no printed marker, because Prof Cheah judged a board out of place in an interface meant to be easy to use. Instead, the arm holds a ball and moves it through 16 to 39 poses. The detector finds the ball in each image, and a rigid fit gives the camera to base transform in one to two minutes. A second step then levels the transform against the table plane. Table 6 shows how the calibration developed.

Date	Method	Fit result	Outcome on the arm
1 Oct	Measured camera mount	Needed a 70 mm hand correction	Picks missed
13 Oct	Ball fit, five fits of 12 to 34 poses	Fit RMS 11.4 to 16.3 mm	Picks still missed
21 Oct	Ball fit after the arm was moved, 39 poses, then levelled (offset 16.8 mm, tilt 2.1 degrees)	Fit RMS 7.3 mm	Reach test missed by 18 to 36 mm, mostly along base x
21 Oct	Reach correction fitted to 4 jogged points (yaw 3.3 degrees plus a shift)	Residual 2.3 to 6.8 mm. Leave-one-out 4 to 12 mm	3 new points needed no jog at a 2 mm step. First full pick and place

Table 6. Calibration history of the camera to base transform.

The reach test found what the fit residual could not. A fit with an RMS of 7.3 mm still missed by up to 36 mm. The likely reason is that the fingers hide part of the orange in every pose, so the detector's centre shifts the same way each time. The correction applied on 21 October is empirical. The next step is a refit with a matte ball that the fingers do not hide, which removes the need for it.

The calibration also sets where the arm may go. The safety envelope is a ring sector around the base, with a radius of 0.40 to 0.70 m, a bearing of -80 to -25 degrees and a height of 0.181 to 0.380 m. The -25 degree edge keeps the arm away from the camera tripod. Every pose either architecture commands is checked against it.

6.5 First pick and place

On 21 October 2026 the new Architecture A ran from speech to a full pick and place for the first time (Figure 7). Table 7 follows the trace of the clean run, which took 31.2 s in all. The spoken instruction was *"Pick up the orange and place it in the red tray."* The pick check saw the orange 43 mm from the TCP, inside the 50 mm limit, so it reported the orange held. The place check saw the orange 162 mm from the tool, so it reported the orange released. I judged both steps correct at the bench.

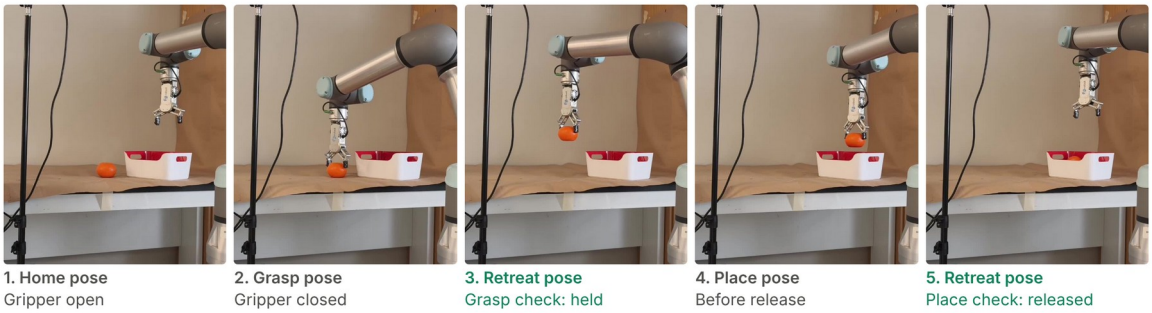


Figure 7. The full pick and place on the real arm on 21 October 2026, from phone video.

The trace also shows a gap. In its final summary the planner said the orange sat about 7 cm from the centre of the tray, possibly on its rim. The place check only tests that the orange left the gripper, and it does not test where the orange landed. An earlier run that day placed the orange correctly, but the drop knocked a light pink container away, and A reported that step as not done. Figure 8 shows the camera view.

Stage	What the trace records	Time
Grounding	Queries "orange" and "red tray"	4.03 s
Perceive	Both found. Orange at (0.551, −0.303, 0.206) m with score 0.79	0.74 s
Plan	One call: pick the orange	1.54 s
Pick and check	Grip closed at 70 mm. Orange 43 mm from the tool, so held	9.25 s
Perceive and plan	Fresh scene, then one call: place in the red tray	2.41 s
Place and check	Released. Orange 162 mm from the tool	6.19 s
Perceive and plan	Fresh scene, then done	4.15 s
Whole run	Operator judged both steps correct	31.2 s

Table 7. Trace of the clean run on 21 October 2026, stage by stage.

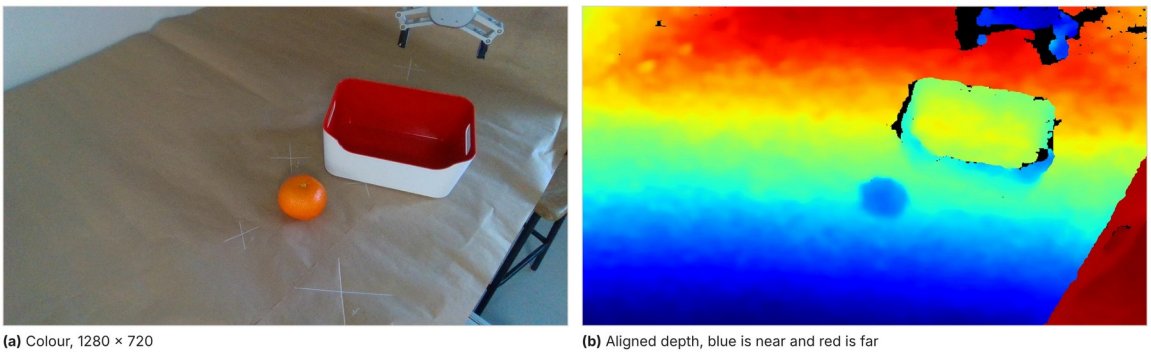


Figure 8. Colour and aligned depth frames from the fixed camera on the real cell. The pen crosses on the table are reach-test targets.

6.6 Problems encountered

Problem	Effect	Response
Lab moved the arm and the camera	All calibration was redone on 21 Oct	Tripod on tape marks, a centre-pixel depth check each session, a fit and a reach test each session
"RTDE input registers are already in use"	The arm could not be driven	Traced to a suspended script still holding the arm. The controller now disconnects on close
A held object read above the height ceiling	The command after a pick was refused	Seen objects are now judged by the sector and the floor only
OWLv2 scored "pink tray" below its 0.3 threshold	The destination was not found	Tier 0 destination changed to the red container, with a dated changelog entry

Table 8. Problems met in Semester 1 and the response to each.

7. Future Work

7.1 Remaining work

- Refit the transform with a matte ball, drop the empirical correction, and validate it with the reach test
- Run the 20-trial Tier 0 run through the harness in December 2026, with success judged at the bench
- Make the place check test where the object landed, and set the release height
- Make perception a tool the planner calls with its own noun phrases, so it can look for new words after a clarification, as Tier 2 needs
- Write the pour skill for Tier 1 and finish the drawer skill for Tier 3
- Record the demonstrations and fine-tune Architecture B in the order of Section 5.2

7.2 Changes to the original requirements

Four changes were made in Semester 1, and each is recorded with a date. Architecture A became a loop on 7 October (Section 4.1). Speech became the only input on 13 October, so no trial can be typed. The Tier 0 objects changed from a red cube and a box to an orange and a red container, on 13 and 26 October, because of what was at hand and what the detector could find. The layouts changed on 29 October from random table coordinates to five fixed crosses. A coordinate cannot be placed by hand at the bench, and after the arm was moved on 21 October the old coordinates fell outside its reach.

7.3 Risks

Risk	Effect	Response
Demonstration coverage	B fails outside the region it was shown	Start in a small workspace, where density matters more than count, then widen
Demonstration quality	Hands in view or uneven motion teach the wrong cues	Jog device instead of hand guiding. Inspect the first five episodes before recording more
GPU memory	12 GB may not fit the SmolVLA fine-tune	Short test run on the first episodes. Then gradient accumulation, gradient checkpointing, an 8-bit optimiser or a frozen vision backbone
Calibration drift	A misses, and B sees a view it was not trained on	Fit and reach test each session. Record demonstrations across the small range of camera positions
Fair comparison	An unequal setting biases the result	Shared modules, a frozen suite, a blind harness and a dated changelog
Learned actions	A predicted chunk can move the arm unsafely	Speed clamp and envelope on every action. Test the e-stop recovery before any trial

Table 9. Risks for Semester 2 and the response to each.

7.4 Schedule

Figure 9 shows the plan to the end of the project. The matte-ball refit and the 20-trial Tier 0 run of A take place in December, after the exams, and the demonstrations start in the same vacation. B reaches Tier 0 by Semester 2 Week 3. The comparison runs through February, and results are frozen at the end of that month. The reports follow on the NTU dates.

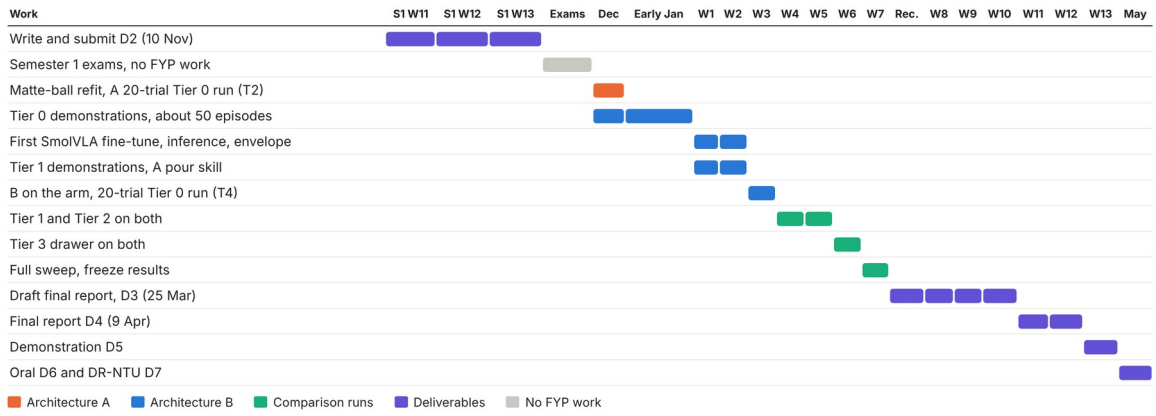


Figure 9. Schedule from Semester 1 Week 11 to the oral presentation in May 2027.

8. Conclusion

Architecture A now runs on the real arm from a spoken command to a full pick and place. Getting there took most of October. The grasps missed for two systematic reasons, a depth point taken from a box average and a transform bias that the fit residual could not show. Both were found from three plain observations and fixed with SAM masks and a reach test after every fit.

The assessment is mixed. The fairness controls are written into code, and the harness has been frozen since August. The calibration still depends on an empirical correction, and the 20-trial Tier 0 run is planned for December 2026. Architecture B has its robot side written but no episode recorded yet. Its training in January and February is the largest risk to the schedule, and the order of work in Section 5.2 is set to meet it.

9. Acknowledging the Use of GAI

Generative AI was used in this project. Claude, a model by Anthropic, helped write and review code and helped draft and edit this report. All design decisions, bench work and results are my own. An LLM is also part of Architecture A, where it is the object of study. [Complete and attach the GAI declaration form from the FYP email.]

References

[1] S. Shan and C. C. Cheah, "A large-language-model-enabled robotic system with application to power cable manipulation," in Proc. CPEEE, 2026, doi: 10.1109/CPEEE69412.2026.11521580.

[2] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence and A. Zeng, "Code as policies: Language model programs for embodied control," in Proc. IEEE Int. Conf. Robotics and Automation (ICRA), 2023. arXiv:2209.07753, 2022.

[3] A. Z. Ren et al., "Robots that ask for help: Uncertainty alignment for large language model planners," in Proc. Conf. Robot Learning (CoRL), PMLR vol. 229, pp. 661-682, 2023.

[4] P. Guo et al., "AgenticLab: A real-world robot agent platform that can see, think, and act," arXiv:2602.01662, Feb. 2026.

[5] S. Lian et al., "BayesianVLA: Bayesian decomposition of vision language action models via latent action queries," arXiv:2601.15197, 2026.

[6] M. Shukor et al., "SmolVLA: A vision-language-action model for affordable and efficient robotics," arXiv:2506.01844, 2025.

[7] M. Minderer, A. Gritsenko and N. Houlsby, "Scaling open-vocabulary object detection," in Advances in Neural Information Processing Systems (NeurIPS), 2023.

[8] A. Kirillov et al., "Segment anything," in Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV), 2023.

[9] K. Black, M. Y. Galliker and S. Levine, "Real-time execution of action chunking flow policies," arXiv:2506.07339, 2025.

[10] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in Proc. Int. Conf. Machine Learning (ICML), 2023.